

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	S Susmitha	Reg.No.:	192571068
Department:	BTech CSE BioSciences	Date:	25.07.2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I S Susmitha (192571068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:


MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 1

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA1202	Computer Architecture

Question 1

A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

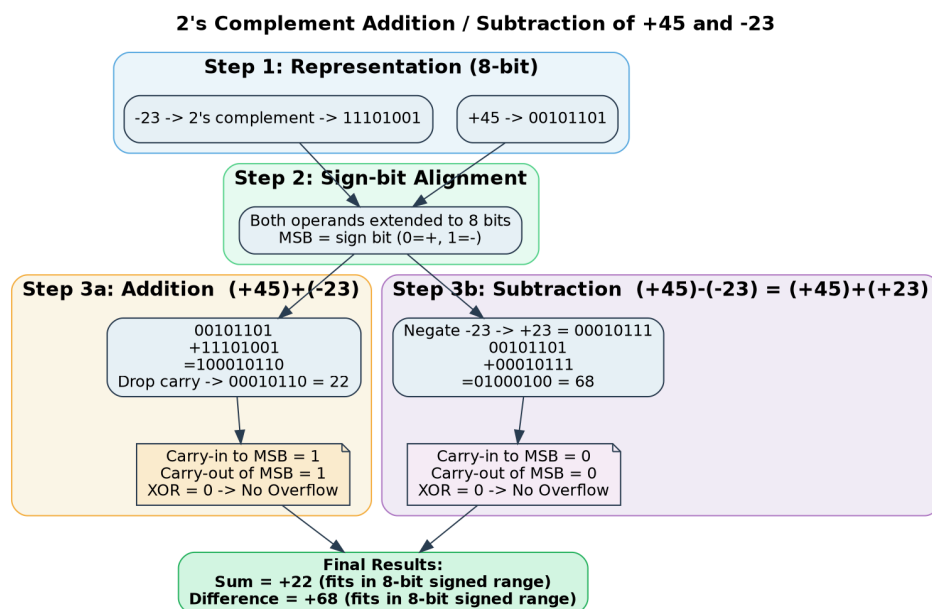
Understanding the Problem Context

The processor stores signed integers in 8-bit two's complement form. We are given two signed numbers, +45 and -23, and must analyse how the ALU performs both addition and subtraction on them. This requires converting both operands to their correct 8-bit two's complement patterns, aligning the sign bits during the arithmetic, carrying out the bit-by-bit binary addition, and finally applying the overflow-detection rule used by real hardware (comparing the carry into the sign bit with the carry out of the sign bit).

Application of Concepts: Two's Complement Arithmetic

Two's complement addition is used because it lets the ALU rely on a single adder circuit for both addition and subtraction — subtraction $A - B$ is implemented as $A + (2\text{'s complement of } B)$, avoiding a separate subtractor circuit. Overflow is detected using the standard rule: if the carry generated into the MSB (sign bit) differs from the carry generated out of the MSB, an overflow has occurred. This is preferred over sign-magnitude arithmetic because it removes the dual representation of zero and simplifies hardware design.

Diagram: 2's Complement Addition and Subtraction Flow



Problem-Solving Approach: Step-by-Step Computation

Representation: $+45 = 00101101$ (8-bit). For -23 , we first write $+23 = 00010111$, invert all bits to get 11101000 , and add 1 to obtain the two's complement 11101001 , which represents -23 .

Addition $(+45) + (-23)$: the two 8-bit patterns are added directly using ordinary binary addition; any carry out of the 8th (final) bit position is discarded because the register width is fixed at 8 bits.

Subtraction $(+45) - (-23)$: subtraction is converted into addition by first negating the second operand. Negating -23 (i.e., taking its two's complement a second time) gives back $+23 = 00010111$, which is then added to $+45$ using the same binary adder.

Code: Simulating 8-bit Two's Complement Addition/Subtraction

The Python program below models an 8-bit two's-complement ALU: it converts both operands to their unsigned 8-bit bit patterns, performs the binary addition, and applies the carry-in/carry-out XOR rule to flag overflow.

```
def to_bin(n, bits=8):
    if n < 0:
        n = (1 << bits) + n
    return format(n, '0{}b'.format(bits))

def twos_complement_add(a, b, bits=8):
    mask = (1 << bits) - 1
    a_u = a & mask
    b_u = b & mask
    sum_u = (a_u + b_u) & mask
    carry_out = 1 if (a_u + b_u) > mask else 0
    # sign bits
    sign_a = (a_u >> (bits-1)) & 1
    sign_b = (b_u >> (bits-1)) & 1
    sign_sum = (sum_u >> (bits-1)) & 1
    # overflow: two operands same sign, result different sign
    overflow = 1 if (sign_a == sign_b and sign_sum != sign_a) else 0
    # signed interpretation
    signed_result = sum_u - (1 << bits) if sum_u >= (1 << (bits-1)) else sum_u
    return a_u, b_u, sum_u, carry_out, overflow, signed_result

print("=== ADDITION: (+45) + (-23) ===")
A = 45
B = -23
a_u, b_u, sum_u, carry_out, overflow, signed_result = twos_complement_add(A, B)
print(f"+45 = {to_bin(45)}")
print(f"-23 = {to_bin(-23)} (2's complement of 23)")
print(f" {to_bin(a_u)}")
print(f"+ {to_bin(b_u)}")
print(f" {'.'*8}")
print(f" {to_bin(sum_u)} (carry out of MSB = {carry_out})")
print(f"Signed result = {signed_result}")
print(f"Overflow = {overflow}")

print()
print("=== SUBTRACTION: (+45) - (-23) = (+45) + (+23) ===")
A2 = 45
B2 = 23 # subtraction done as addition of 2's complement of (-23), i.e. +23
a_u2, b_u2, sum_u2, carry_out2, overflow2, signed_result2 = twos_complement_add(A2, B2)
print(f"+45 = {to_bin(45)}")
```

```

print(f"+23 = {to_bin(23)} (2's complement negation of -23 gives +23)")
print(f" {to_bin(a_u2)}")
print(f"+ {to_bin(b_u2)}")
print(f" {'-'*8}")
print(f" {to_bin(sum_u2)} (carry out of MSB = {carry_out2})")
print(f"Signed result = {signed_result2}")
print(f"Overflow = {overflow2}")

```

Output

```

>>>
===== RESTART: C:/Users/Susmitha/Downloads/CSA1202 Assessment/CO2 AT1 Q1.py =====
=== ADDITION: (+45) + (-23) ===
+45  = 00101101
-23  = 11101001 (2's complement of 23)
      00101101
+ 11101001
-----
      00010110 (carry out of MSB = 1)
Signed result = 22
Overflow = 0

=== SUBTRACTION: (+45) - (-23) = (+45) + (+23) ===
+45  = 00101101
+23  = 00010111 (2's complement negation of -23 gives +23)
      00101101
+ 00010111
-----
      01000100 (carry out of MSB = 0)
Signed result = 68
Overflow = 0

```

The simulation confirms the manual working: $(+45) + (-23)$ gives $00010110 = 22$, with the carry into and out of the sign bit both equal to 1, so no overflow occurs. $(+45) - (-23)$, computed as $(+45) + (+23)$, gives $01000100 = 68$, with both carries equal to 0, again indicating no overflow.

Solution Summary and Justification

Both results fit safely within the 8-bit signed range of -128 to $+127$, so the equal carry-in/carry-out pairs correctly signal no overflow in either case. This confirms that representing -23 via two's complement and reusing the same binary adder for both addition and subtraction (by negating the subtrahend) is sufficient for the processor to compute both $+22$ and $+68$ accurately, while the carry-comparison rule gives it a simple, purely bitwise way to detect when a result would exceed the representable range.

Question 2

A high-performance computing system replaces a ripple carry adder with a 4-bit carry lookahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

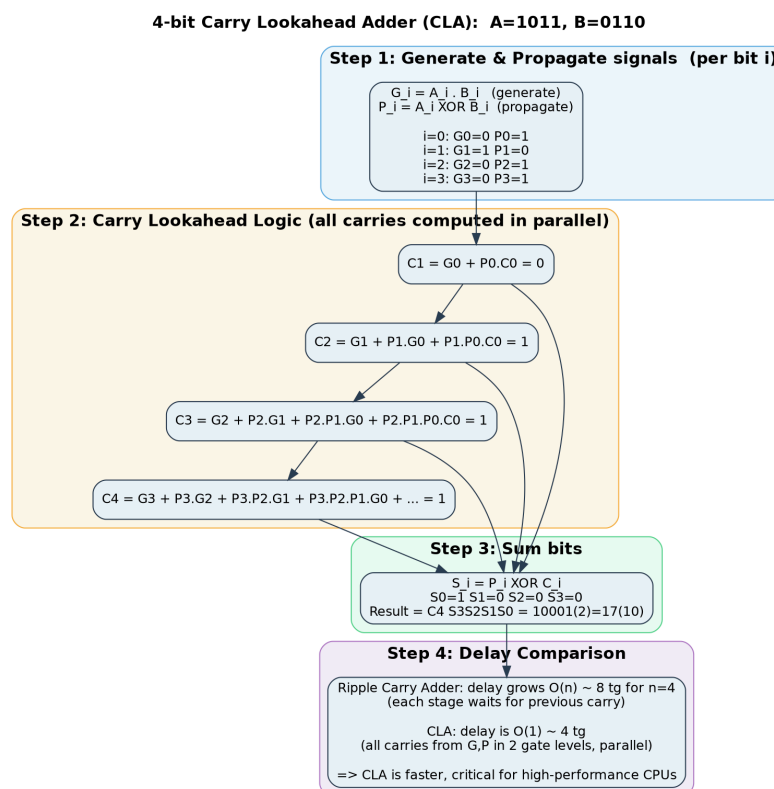
Understanding the Problem Context

A ripple carry adder (RCA) computes each carry only after the previous stage's carry is available, so the total delay grows linearly with the number of bits. The problem requires analysing how a 4-bit Carry Lookahead Adder removes this dependency by pre-computing all carries directly from the input bits using generate (G) and propagate (P) signals, and then comparing the resulting speed against a ripple carry adder for the chosen inputs, $A = 1011$ and $B = 0110$.

Application of Concepts: Generate and Propagate Signals

The Carry Lookahead method expresses every carry C_{i+1} purely as a Boolean function of the generate/propagate signals of all lower bits and the initial carry-in, so every carry can be evaluated in parallel by two levels of AND-OR gates instead of waiting for a rippling chain. For each bit position i , the generate signal is $G_i = A_i \cdot B_i$ (a carry is generated regardless of the incoming carry), and the propagate signal is $P_i = A_i \text{ XOR } B_i$ (an incoming carry would be passed through).

Diagram: CLA Generate/Propagate and Carry Lookahead Logic



Problem-Solving Approach: Computing Carries in Advance

Using G_i and P_i , the lookahead carries are expressed directly in terms of the primary inputs: $C1 = G0 + P0 \cdot C0$; $C2 = G1 + P1 \cdot G0 + P1 \cdot P0 \cdot C0$; $C3 = G2 + P2 \cdot G1 + P2 \cdot P1 \cdot G0 + P2 \cdot P1 \cdot P0 \cdot C0$; $C4 = G3 + P3 \cdot G2 + P3 \cdot P2 \cdot G1 + P3 \cdot P2 \cdot P1 \cdot G0 + P3 \cdot P2 \cdot P1 \cdot P0 \cdot C0$. Because none of these expressions depends on a previous stage's computed carry, all four carries can be generated simultaneously once G and P are known, which is exactly what removes the ripple delay.

Code: Computing G, P, Carries and Delay Comparison

The program below computes G_i , P_i and the four lookahead carries for $A = 1011$ (11) and $B = 0110$ (6), forms the sum, and prints a simplified gate-delay comparison between ripple carry and carry-lookahead adders.

```
def bits(n, width=4):
    return [ (n >> i) & 1 for i in range(width) ] # bit0..bit3 (LSB first)

A = 0b1011 # 11
B = 0b0110 # 6
C0 = 0

a = bits(A)
b = bits(B)

G = [a[i] & b[i] for i in range(4)]
P = [a[i] ^ b[i] for i in range(4)]

C = [0]*5
C[0] = C0
C[1] = G[0] | (P[0] & C[0])
C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & C[0])
C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] & P[1] & P[0] & C[0])
C[4] = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) | (P[3] & P[2] & P[1] & G[0]) | (P[3] & P[2] & P[1] & P[0] & C[0])

S = [P[i] ^ C[i] for i in range(4)]

print(f"A = {A:04b} ({A}), B = {B:04b} ({B}), Cin = {C0}")
print("i : a_i b_i G_i P_i C_{i+1}")
for i in range(4):
    print(f"i : {a[i]} {b[i]} {G[i]} {P[i]} {C[i+1]}")

sum_val = sum(S[i] << i for i in range(4))
result = sum_val + (C[4] << 4)
print()
print("Sum bits S3 S2 S1 S0 =", ".join(str(S[i]) for i in reversed(range(4))))")
print("Carry out C4 =", C[4])
print(f"Result = C4 S3S2S1S0 = {C[4]} {" ".join(str(S[i]) for i in reversed(range(4)))} = {result} (decimal)")
print(f"Verification: {A} + {B} = {A+B}")

# Ripple carry timing comparison (symbolic, gate delay = 1 unit "tg")
print()
print("==== Delay comparison (in gate delays, tg) ====")
n = 4
ripple_delay = 2*n # each full adder ~2 gate delays for carry propagation chain (simplified teaching model)
cla_delay = 4 # constant: 2 levels for G/P + 2 levels for carry lookahead logic, independent of n
print(f"Ripple Carry Adder (n={n}): worst-case carry propagation delay ~ {ripple_delay} tg (grows with n)")
print(f"Carry Lookahead Adder (n={n}): delay ~ {cla_delay} tg (independent of n, computed in parallel)")
```

Output

```
>>> ===== RESTART: C:/Users/Susmitha/Downloads/CSA1202 Assessment/CO2 AT1 Q2.py =====
A = 1011 (11), B = 0110 (6), Cin = 0
i : a_i b_i G_i P_i C_{i+1}
0 : 1 0 0 1 0
1 : 1 1 1 0 1
2 : 0 1 0 1 1
3 : 1 0 0 1 1

Sum bits S3 S2 S1 S0 = 0001
Carry out C4 = 1
Result = C4 S3S2S1S0 = 10001 = 17 (decimal)
Verification: 11 + 6 = 17

=== Delay comparison (in gate delays, tg) ===
Ripple Carry Adder (n=4): worst-case carry propagation delay ~ 8 tg (grows with n)
Carry Lookahead Adder (n=4): delay ~ 4 tg (independent of n, computed in parallel)
```

The simulation gives $G_0=0$, $G_1=1$, $G_2=0$, $G_3=0$ and $P_0=1$, $P_1=0$, $P_2=1$, $P_3=1$, leading to carries $C_1=0$, $C_2=1$, $C_3=1$, $C_4=1$. The sum bits $S_3S_2S_1S_0 = 0001$ with carry-out $C_4=1$ give the result $10001_2 = 17_{10}$, correctly matching $11 + 6 = 17$.

Solution Summary and Justification

A ripple carry adder must wait for each stage's carry before the next stage can settle, giving a worst-case delay that grows in proportion to the number of bits (about $2n$ gate delays for an n -bit adder — roughly 8 t_g here). The carry lookahead adder computes every carry directly from G and P signals in a fixed 2-level AND-OR structure, so its delay stays roughly constant at about 4 t_g regardless of n . For a 4-bit adder this already nearly halves the delay, and the advantage grows sharply for wider registers (32-bit, 64-bit), which is why CLA logic — or hybrid block-CLA structures — is used inside modern processor ALUs to keep addition off the critical path of the clock cycle.

Question 3

In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

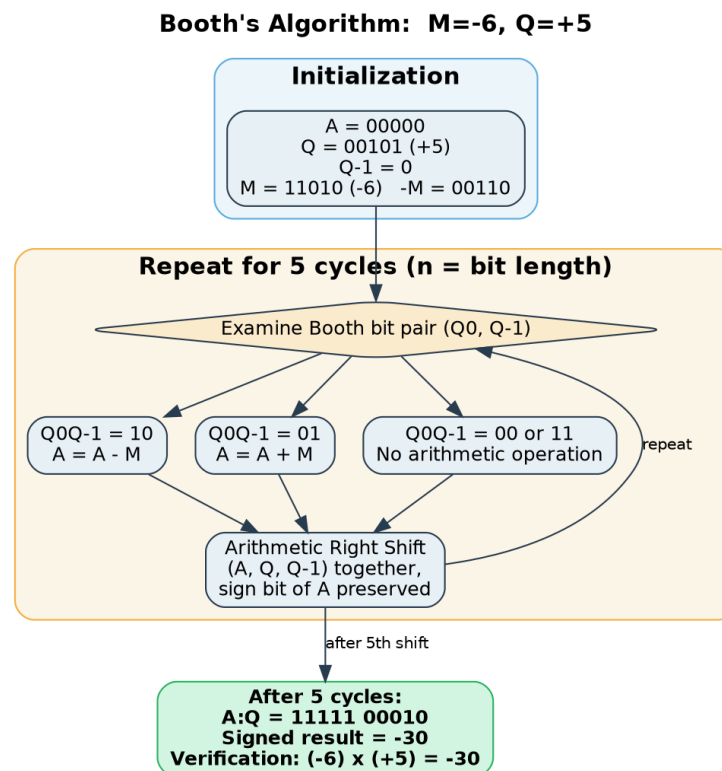
Understanding the Problem Context

Booth's algorithm multiplies two signed binary numbers directly in two's complement form, without converting operands to magnitude-and-sign first. The task is to multiply -6 and $+5$ and to trace the role of the accumulator (A), the multiplier register (Q), the extra Booth bit (Q_{-1}), and the arithmetic right-shift that combines A, Q and Q_{-1} after every cycle.

Application of Concepts: Booth's Algorithm

Booth's algorithm is used because it directly handles negative multiplicands and multipliers without a separate sign-correction step, and it reduces the number of add/subtract operations whenever the multiplier contains runs of consecutive 1s or 0s (each such run collapses into a single add or subtract at its boundary instead of one addition per bit). This makes it considerably more efficient than repeated shift-and-add multiplication for signed operands.

Diagram: Booth's Algorithm Cycle



Problem-Solving Approach: Step-by-Step Trace

Both operands are represented in 5-bit two's complement to safely hold -6 : M (multiplicand) = $-6 = 11010$, and its negation $-M = 00110$. Q (multiplier) = $+5 = 00101$, with Q_{-1} initialised to 0 and the accumulator A initialised to 0. On each of the 5 cycles, the pair (Q0, Q_{-1}) is inspected: '10' triggers $A = A - M$, '01' triggers $A = A + M$,

and '00' or '11' perform no arithmetic. After the arithmetic step, the combined register (A, Q, Q-1) is shifted one place to the right arithmetically, preserving the sign bit of A. After all 5 cycles, the concatenation of A and Q gives the final signed product.

Code: Simulating Booth's Algorithm

The Python program below implements Booth's algorithm exactly as specified (5-bit registers, arithmetic shifts preserving sign) and prints the full cycle-by-cycle trace of A, Q and Q-1.

```
def to_bin(n, bits):
    if n < 0:
        n = (1 << bits) + n
    return format(n & ((1<<bits)-1), '0{}b'.format(bits))

def arith_shift_right(A, Q, Qm1, bits):
    combined_sign = (A >> (bits-1)) & 1
    new_Qm1 = Q & 1
    Q = ((Q >> 1) | ((A & 1) << (bits-1))) & ((1<<bits)-1)
    A = ((A >> 1) | (combined_sign << (bits-1))) & ((1<<bits)-1)
    return A, Q, new_Qm1

def booth_multiply(M_val, Q_val, bits=5):
    mask = (1<<bits)-1
    M = M_val & mask
    negM = (-M_val) & mask
    A = 0
    Q = Q_val & mask
    Qm1 = 0
    steps = []
    for i in range(bits):
        q0 = Q & 1
        action = ""
        if q0 == 1 and Qm1 == 0:
            A = (A - M) & mask
            action = "Q0Q-1=10 -> A = A - M"
        elif q0 == 0 and Qm1 == 1:
            A = (A + M) & mask
            action = "Q0Q-1=01 -> A = A + M"
        else:
            action = f"Q0Q-1={q0}{Qm1} -> no operation"
        steps.append((i+1, action, to_bin(A,bits), to_bin(Q,bits), Qm1, "before shift"))
        A, Q, Qm1 = arith_shift_right(A, Q, Qm1, bits)
        steps.append((i+1, "Arithmetic right shift (A,Q,Q-1)", to_bin(A,bits), to_bin(Q,bits), Qm1, "after shift"))
    # final result = A:Q (2*bits length), interpret as signed
    result_bits = bits*2
    combined = (A << bits) | Q
    if combined >= (1 << (result_bits-1)):
        combined -= (1 << result_bits)
    return steps, A, Q, combined

M_val = -6
Q_val = 5
bits = 5
steps, A_final, Q_final, result = booth_multiply(M_val, Q_val, bits)

print(f"M (multiplicand) = {M_val} = {to_bin(M_val,bits)} ( {bits}-bit 2's complement)")
print(f"Q (multiplier) = {Q_val} = {to_bin(Q_val,bits)}")
```

```

print(f"-M = {to_bin(-M_val, bits)}")
print()
print(f"{'Cycle':<6} {'Operation':<32} {'A':<8} {'Q':<8} {'Q-1':<5}")
for s in steps:
    print(f"{s[0]:<6} {s[1]:<32} {s[2]:<8} {s[3]:<8} {s[4]:<5}")

print()
print(f"Final A:Q = {A_final} : {Q_final} -> combined = {result}")
print(f"Verification: {M_val} x {Q_val} = {M_val*Q_val}")

```

Output

```

>>> ===== RESTART: C:/Users/Susmitha/Downloads/CSA1202 Assessment/CO2 AT1 Q3.py
M (multiplicand) = -6 = 11010 (5-bit 2's complement)
Q (multiplier) = 5 = 00101
-M = 00110

Cycle Operation A Q Q-1
1 Q0Q-1=10 -> A = A - M 00110 00101 0
1 Arithmetic right shift (A,Q,Q-1) 00011 00010 1
2 Q0Q-1=01 -> A = A + M 11101 00010 1
2 Arithmetic right shift (A,Q,Q-1) 11110 10001 0
3 Q0Q-1=10 -> A = A - M 00100 10001 0
3 Arithmetic right shift (A,Q,Q-1) 00010 01000 1
4 Q0Q-1=01 -> A = A + M 11100 01000 1
4 Arithmetic right shift (A,Q,Q-1) 11110 00100 0
5 Q0Q-1=00 -> no operation 11110 00100 0
5 Arithmetic right shift (A,Q,Q-1) 11111 00010 0

Final A:Q = 31 : 2 -> combined = -30
Verification: -6 x 5 = -30

```

Tracing the 5 cycles gives the operation sequence: subtract, shift; add, shift; subtract, shift; add, shift; no-op, shift — ending with A:Q = 11111 00010, which as a signed 10-bit value equals -30 , exactly matching $(-6) \times (+5) = -30$.

Solution Summary and Justification

The final product -30 confirms that Booth's algorithm correctly handles the negative multiplicand purely through arithmetic shifts and conditional add/subtract steps, with no separate sign-handling logic required. Compared with plain repeated-addition multiplication (which would need up to 5 separate additions), Booth's algorithm here needed only 2 additions and 2 subtractions across the 5 cycles, illustrating its efficiency whenever the multiplier bits are not all identical runs of maximum length.

Question 4

A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

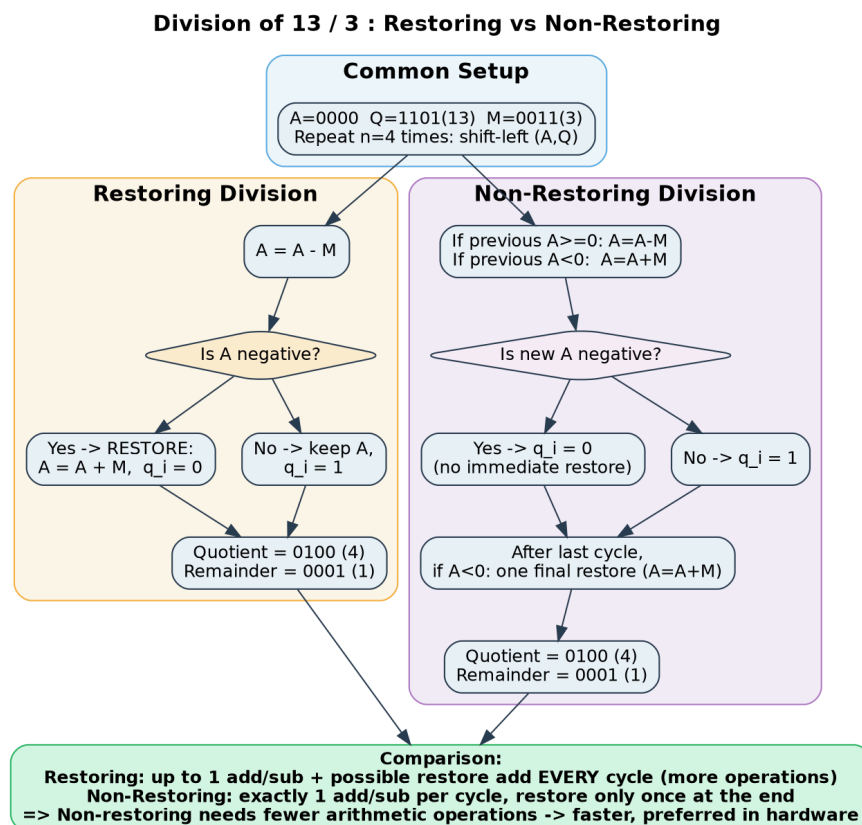
Understanding the Problem Context

Both restoring and non-restoring division compute a quotient and remainder for $13 \div 3$ using repeated shift-and-subtract cycles on a combined accumulator–quotient register pair, but they differ in how they react to a negative partial remainder. The task requires tracing both algorithms step-by-step over 4 cycles (since the dividend fits in 4 bits) and comparing their efficiency.

Application of Concepts: Restoring vs Non-Restoring Division

Restoring division is the more intuitive method — after every trial subtraction it checks the sign and immediately undoes ('restores') the subtraction if the result went negative. Non-restoring division is the more hardware-efficient alternative because it avoids this restoring addition on every cycle, instead carrying the negative remainder forward and compensating with an addition on the following cycle, needing only one arithmetic operation per cycle plus at most one final correction.

Diagram: Restoring vs Non-Restoring Division Flow



Problem-Solving Approach: Step-by-Step Trace

For both methods the registers are initialised as $A = 0000$, $Q = 1101$ (13), $M = 0011$ (3), and 4 cycles are performed (since the dividend uses 4 bits). Each cycle begins with a left shift of the combined (A, Q) pair. In

restoring division, $A = A - M$ is then computed; if the result is negative the quotient bit is set to 0 and M is added back to restore A , otherwise the quotient bit is set to 1 and A is kept as is. In non-restoring division, the operation performed depends on the sign of A from the previous cycle: $A = A - M$ if A was non-negative, or $A = A + M$ if A was negative; the quotient bit is set to 1 if the new A is non-negative and 0 otherwise, with no immediate restoring step. After the 4th cycle, non-restoring division applies one final correcting addition only if the remainder is still negative.

Code: Simulating Restoring and Non-Restoring Division

The program below implements both algorithms with proper 4-bit two's-complement register handling and prints the cycle-by-cycle register states for $13 \div 3$.

```
def to_bin(n, bits):
    if n < 0:
        n = (1<<bits) + n
    return format(n & ((1<<bits)-1), '0{}b'.format(bits))

def to_signed(u, bits):
    if u & (1 << (bits-1)):
        return u - (1 << bits)
    return u

def restoring_division(dividend, divisor, n=4):
    mask = (1<<n)-1
    A = 0
    Q = dividend & mask
    M = divisor & mask
    print("--- Restoring Division: 13 / 3 (n=4 bit registers) ---")
    print(f"Initial: A={to_bin(A,n)} Q={to_bin(Q,n)} M={to_bin(M,n)}")
    for i in range(n):
        combined = ((A & mask) << n) | Q
        combined = (combined << 1) & ((1 << (2*n)) - 1)
        A = (combined >> n) & mask
        Q = combined & mask
        A = (A - M) & mask
        if to_signed(A, n) < 0:
            print(f"Step {i+1}: shift-left -> A-M = {to_signed(A,n)} (negative) -> RESTORE, q{n-1-i}=0")
            A = (A + M) & mask
        else:
            Q = Q | 1
            print(f"Step {i+1}: shift-left -> A-M = {to_signed(A,n)} (>=0) -> keep, q{n-1-i}=1")
            print(f"    A={to_bin(A,n)} Q={to_bin(Q,n)}")
    print(f"RESULT -> Quotient Q = {to_bin(Q,n)} = {Q}, Remainder A = {to_bin(A,n)} = {to_signed(A,n)}")
    return Q, to_signed(A,n)

def non_restoring_division(dividend, divisor, n=4):
    mask = (1<<n)-1
    A = 0
    Q = dividend & mask
    M = divisor & mask
    print()
    print("--- Non-Restoring Division: 13 / 3 (n=4 bit registers) ---")
    print(f"Initial: A={to_bin(A,n)} Q={to_bin(Q,n)} M={to_bin(M,n)}")
    for i in range(n):
        combined = ((A & mask) << n) | Q
        combined = (combined << 1) & ((1 << (2*n)) - 1)
```

```

A = (combined >> n) & mask
Q = combined & mask
if to_signed(A, n) >= 0:
    A = (A - M) & mask
    op = f"A-M = {to_signed(A,n)}"
else:
    A = (A + M) & mask
    op = f"A+M = {to_signed(A,n)}"
if to_signed(A, n) < 0:
    print(f"Step {i+1}: shift-left -> {op} (negative) -> q{n-1-i}=0")
else:
    Q = Q | 1
    print(f"Step {i+1}: shift-left -> {op} (>=0) -> q{n-1-i}=1")
print(f"    A={to_bin(A,n)} Q={to_bin(Q,n)}")
if to_signed(A, n) < 0:
    A = (A + M) & mask
    print(f"Final A negative -> one restoring add: A = A + M = {to_signed(A,n)}")
print(f"RESULT -> Quotient Q = {to_bin(Q,n)} = {Q}, Remainder A = {to_bin(A,n)} = {to_signed(A,n)}")
return Q, to_signed(A,n)

```

```

Qr, Rr = restoring_division(13, 3)
Qn, Rn = non_restoring_division(13, 3)
print()
print(f"Verification: 13 = 3*{13//3} + {13%3}")
print(f"Restoring result: Q={Qr}, R={Rr} {'MATCH' if (Qr,Rr)==(13//3,13%3) else 'MISMATCH'}")
print(f"Non-Restoring result: Q={Qn}, R={Rn} {'MATCH' if (Qn,Rn)==(13//3,13%3) else 'MISMATCH'}")

```

Output

```

>>>
===== RESTART: C:/Users/Susmitha/Downloads/CSA1202 Assessment/CO2 AT1 Q4.py
--- Restoring Division: 13 / 3 (n=4 bit registers) ---
Initial: A=0000 Q=1101 M=0011
Step 1: shift-left -> A-M = -2 (negative) -> RESTORE, q3=0
      A=0001 Q=1010
Step 2: shift-left -> A-M = 0 (>=0) -> keep, q2=1
      A=0000 Q=0101
Step 3: shift-left -> A-M = -3 (negative) -> RESTORE, q1=0
      A=0000 Q=1010
Step 4: shift-left -> A-M = -2 (negative) -> RESTORE, q0=0
      A=0001 Q=0100
RESULT -> Quotient Q = 0100 = 4, Remainder A = 0001 = 1

--- Non-Restoring Division: 13 / 3 (n=4 bit registers) ---
Initial: A=0000 Q=1101 M=0011
Step 1: shift-left -> A-M = -2 (negative) -> q3=0
      A=1110 Q=1010
Step 2: shift-left -> A+M = 0 (>=0) -> q2=1
      A=0000 Q=0101
Step 3: shift-left -> A-M = -3 (negative) -> q1=0
      A=1101 Q=1010
Step 4: shift-left -> A+M = -2 (negative) -> q0=0
      A=1110 Q=0100
Final A negative -> one restoring add: A = A + M = 1
RESULT -> Quotient Q = 0100 = 4, Remainder A = 0001 = 1

Verification: 13 = 3*4 + 1
Restoring result: Q=4, R=1 MATCH
Non-Restoring result: Q=4, R=1 MATCH

```

Both algorithms converge to the same final registers: Quotient $Q = 0100$ (4) and Remainder $A = 0001$ (1), matching the expected result $13 = 3 \times 4 + 1$.

Solution Summary and Justification

Restoring division needed a restoring addition in 3 of its 4 cycles (cycles 1, 3 and 4), each restoring step being an extra addition beyond the trial subtraction — so up to 2 arithmetic operations occurred in those cycles. Non-restoring division performed exactly one addition or subtraction per cycle (4 operations total) plus a single optional correction at the very end, giving it a lower and more predictable worst-case operation count. This is why non-restoring division is generally preferred in hardware dividers: its operation count per cycle is fixed and independent of the sign pattern of intermediate remainders, which simplifies control logic and shortens the critical path compared with restoring division's conditional extra addition.

Question 5

A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Understanding the Problem Context

The IEEE 754 standard represents real numbers using three fields — sign, biased exponent, and mantissa (fraction) — in both 32-bit single precision and 64-bit double precision formats. The task requires representing -13.25 in both formats and then analysing how floating-point addition is carried out between two such numbers, including the normalisation, exponent-alignment, and rounding issues involved.

Application of Concepts: IEEE 754 Encoding

The standard IEEE 754 encoding procedure converts the decimal magnitude to binary, normalises it to the form $1.xxxx \times 2^e$, then splits the sign, biased exponent ($e + \text{bias}$, where bias = 127 for single precision and 1023 for double precision), and mantissa bits into their respective fields. This is the universally implemented hardware/software standard for real-number arithmetic, ensuring portability and well-defined rounding and overflow behaviour across all IEEE-754-compliant systems.

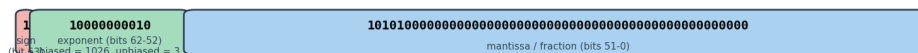
Diagram: IEEE 754 Bit-Field Layout for -13.25

IEEE 754 Representation of -13.25

Single Precision (32-bit)



Double Precision (64-bit)



Derivation:

13.25 (decimal) $\rightarrow$ 1101.01 (binary) $\rightarrow$ normalize: 1.10101×2^3
Sign = 1 (negative) Exponent (biased) = $3 + 127 = 130$ (single) / $3 + 1023 = 1026$ (double)
Mantissa = fractional part after leading 1. = 10101 followed by zero-padding

Problem-Solving Approach: Step-by-Step Encoding

First, 13.25 is converted to binary: the integer part $13 = 1101(2)$ and the fractional part $0.25 = .01(2)$, giving $1101.01(2)$. Normalising this yields 1.10101×2^3 . The sign bit is 1 since the number is negative. The unbiased exponent is 3, so the biased exponent is $3 + 127 = 130$ for single precision (10000010 in 8 bits) and $3 + 1023 = 1026$ for double precision (1000000010 in 11 bits). The mantissa stores the bits after the leading '1.' — namely 10101 — padded with zeros to fill the remaining 23 bits (single) or 52 bits (double).

Code: Deriving IEEE 754 Fields and Floating-Point Addition

The Python program below uses IEEE-754-compliant bit packing (via the struct module) to derive the exact sign, exponent, and mantissa fields for -13.25 in both formats, and then demonstrates floating-point addition with a second operand, $+6.5$.

```
import struct

def float_to_ieee_bits(value, fmt):
    if fmt == 'single':
        packed = struct.pack('>f', value)
        bits = ''.join(f'{b:08b}' for b in packed)
        exp_bits, mant_bits = 8, 23
        bias = 127
    else:
        packed = struct.pack('>d', value)
        bits = ''.join(f'{b:08b}' for b in packed)
        exp_bits, mant_bits = 11, 52
        bias = 1023
    sign = bits[0]
    exponent = bits[1:1+exp_bits]
    mantissa = bits[1+exp_bits:1+exp_bits+mant_bits]
    return sign, exponent, mantissa, int(exponent,2)-bias

value = -13.25
print(f'Number: {value}')
print()
print("Step 1: Convert magnitude to binary")
print("13 = 1101(base2), 0.25 = .01(base2) -> 13.25 = 1101.01(base2)")
print("Normalized: 1101.01 = 1.10101 x 2^3")
print()

for fmt in ['single', 'double']:
    sign, exponent, mantissa, real_exp = float_to_ieee_bits(value, fmt)
    label = "Single Precision (32-bit)" if fmt=='single' else "Double Precision (64-bit)"
    print(f'--- {label} ---')
    print(f'Sign = {sign} (negative)')
    print(f'Exponent = {exponent} (biased, = {int(exponent,2)}, unbiased = {real_exp})')
    print(f'Mantissa = {mantissa}')
    print(f'Full bit pattern = {sign} {exponent} {mantissa}')
    print()

print("=== Floating-Point Addition Example: (-13.25) + (6.5) ===")
a = -13.25
b = 6.5
result = a + b
print(f'{a} = -1.10101 x 2^3')
print(f'{b} = 1.101 x 2^2')
print("Align exponents (shift 6.5's mantissa right by 1 to match 2^3):")
print("6.5 = 0.1101 x 2^3 (after alignment, precision consideration)")
print(f'Sum in decimal check = {a} + {b} = {result}')
print(f'Result = {result} = binary {bin(int(abs(result)))[2:] if result==int(result) else 'non-integer, has fraction'}")

# show actual bits of result for verification
sign_r, exp_r, mant_r, real_exp_r = float_to_ieee_bits(result, 'single')
print(f'IEEE-754 single of result ({result}): sign={sign_r} exponent={exp_r} mantissa={mant_r}')
```

Output

```
>>>===== RESTART: C:/Users/Susmitha/Downloads/CSA1202 Assessment/CO2 AT1 Q5.py
Number: -13.25

Step 1: Convert magnitude to binary
13 = 1101(base2), 0.25 = .01(base2)  -> 13.25 = 1101.01(base2)
Normalized: 1101.01 = 1.10101 x 2^3

--- Single Precision (32-bit) ---
Sign      = 1 (negative)
Exponent  = 10000010 (biased, = 130, unbiased = 3)
Mantissa  = 101010000000000000000000
Full bit pattern = 1 10000010 101010000000000000000000

--- Double Precision (64-bit) ---
Sign      = 1 (negative)
Exponent  = 10000000010 (biased, = 1026, unbiased = 3)
Mantissa  = 1010100000000000000000000000000000000000000000000000000
Full bit pattern = 1 10000000010 10101000000000000000000000000000000000000000000000000000

=== Floating-Point Addition Example: (-13.25) + (6.5) ===
-13.25 = -1.10101 x 2^3
6.5 = 1.101 x 2^2
Align exponents (shift 6.5's mantissa right by 1 to match 2^3):
6.5 = 0.1101 x 2^3 (after alignment, precision consideration)
Sum in decimal check = -13.25 + 6.5 = -6.75
Result = -6.75 = binary non-integer, has fraction
IEEE-754 single of result (-6.75): sign=1 exponent=10000001 mantissa=101100000000000000000000
```

Single precision: sign = 1, exponent = 10000010 (130 biased, unbiased 3), mantissa = 101010000000000000000000. Double precision: sign = 1, exponent = 10000000010 (1026 biased, unbiased 3), mantissa = 1010100. Both encodings correctly reconstruct -13.25 when decoded.

Solution Summary and Justification

For the addition example $(-13.25) + (+6.5)$, the two exponents (2^3 and 2^2) must first be equalised before the mantissas can be added directly; this is done by right-shifting the smaller-exponent operand's mantissa by the exponent difference (1 bit here), which can discard low-order mantissa bits and is the root cause of precision loss in floating-point addition when operand magnitudes differ substantially. After alignment, the mantissas are combined (effectively subtracted here, since the signs differ), and the result is re-normalised so that the leading bit is again '1', with the exponent adjusted accordingly and the final mantissa rounded to fit the available bits. The computed result -6.75 matches the direct decimal check, confirming the alignment-add-normalise-round sequence was carried out correctly.